



Go (Golang)

The basics...

Mikael Berglund 2024-03-21

Index



- About
- For what
- Pros and cons
- Install
- Hello world
- Debug
- Build
- Importing 3rd party packages
- Variables
- Data types
- Packages
- Private/Public
- Go routines
- Panic
- Defer (try/catch)
- Recover
- Normal error handling
- Channels
- Select
- Waitgroup
- Leakage
- Context
- Embed
- Inheritance
- Polymorphism
- Other

About

- Created by Google 2007
- Sometimes called Golang
- Syntax that builds on C
- Statically typed
- Garbage collecting
- Parallell thread execution
- Code in VsCode



For what?



- Programs that must scale
- Programs that need effective thread execution
- Development on one OS, build to several
- Cloud-programs
- Performance critical applications
- Applications with limited memory resources

Pros and cons



Pros

- Install/develop on Windows, Mac, Linux
- Cross compiling (builds everything to one binary)
- Known for its incredible performance (efficient runtime, garbage collection and compiled to machine code)
- Known for its great memory efficiency (small footprint)
- Very easy thread handling
- Easy to learn (few commands)
- Clean syntax

Pros and cons



Cons

- Not object oriented (no classes, but structs)
- No default values on function arguments, `func test(a:= "")` does not work.
- No overloading of functions (two functions, with different amount of arguments, cannot have the same name)
- If you import a package that uses, for example, the package name “nats”, you cannot also declare a function in your code called “nats”.
- You cannot compile, or run, your program if you have any declared variable that is not used.
- By default when saving, auto format of your code is applied. This means that all unused imports will just be deleted. It means that you cannot save your work until you have completed using your imports.
- No try/catch

Install



- Download here: <https://go.dev/dl/>
- Follow instructions: <https://go.dev/doc/install>

Hello world



```
$ mkdir hello  
$ cd hello  
$ go mod init example/hello
```

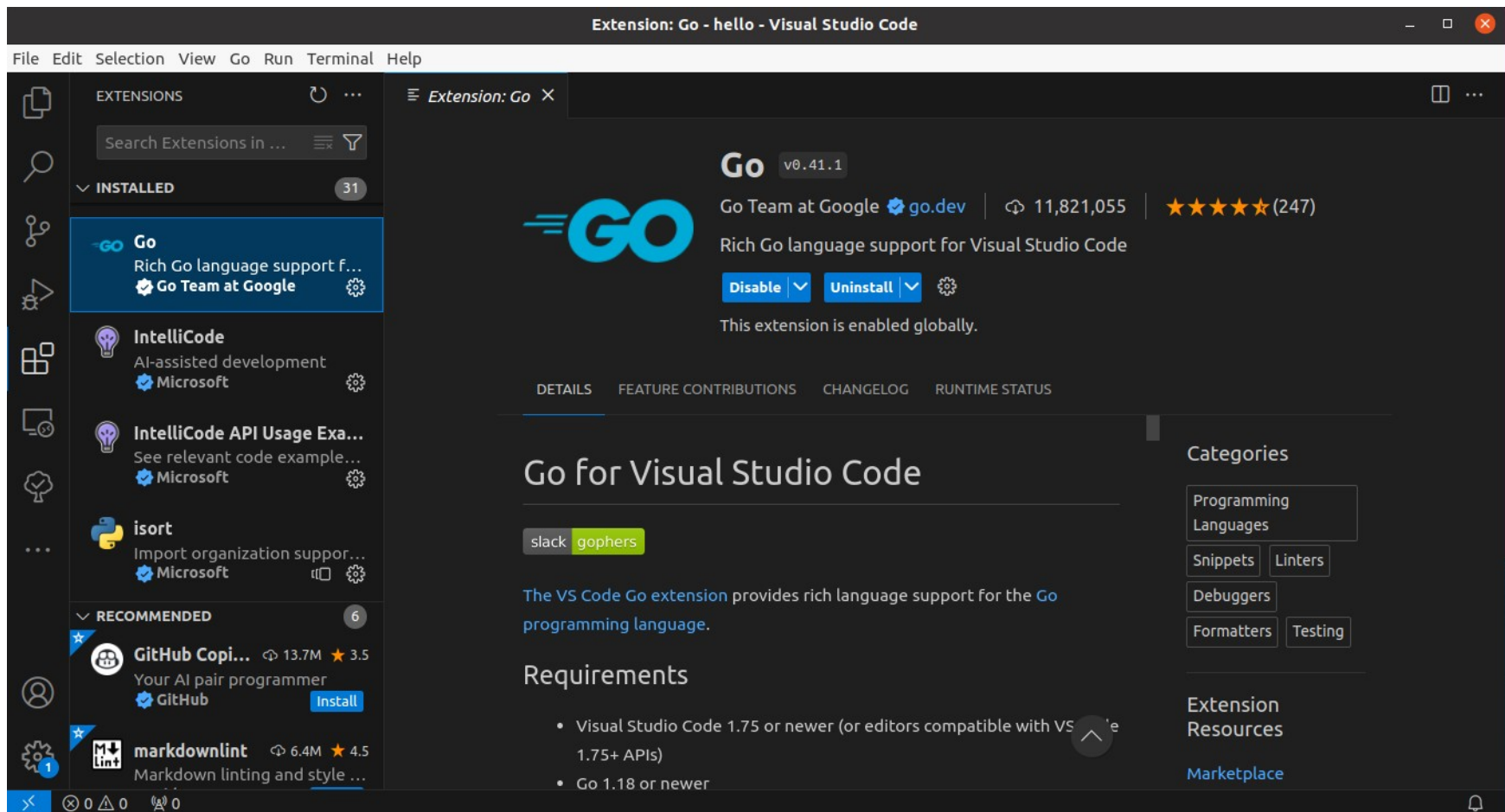
When you execute the last command, it will create a go.mod file in your hello folder. The go.mod file will contain information about which 3rd party modules you import into your program, together with their version.

After “go mod init” you shall enter the name that you want for your own module, in this case “**example/hello**”. Normally you will save your module to a repo like github.com/hello. In that case you would name your module after this, like: go mod init github.com/hello. Others could then import your code with this name.

```
$ cat go.mod  
module example/hello  
  
go 1.21.1
```

Hello world

- Open VsCode and install the Go extension



Hello world



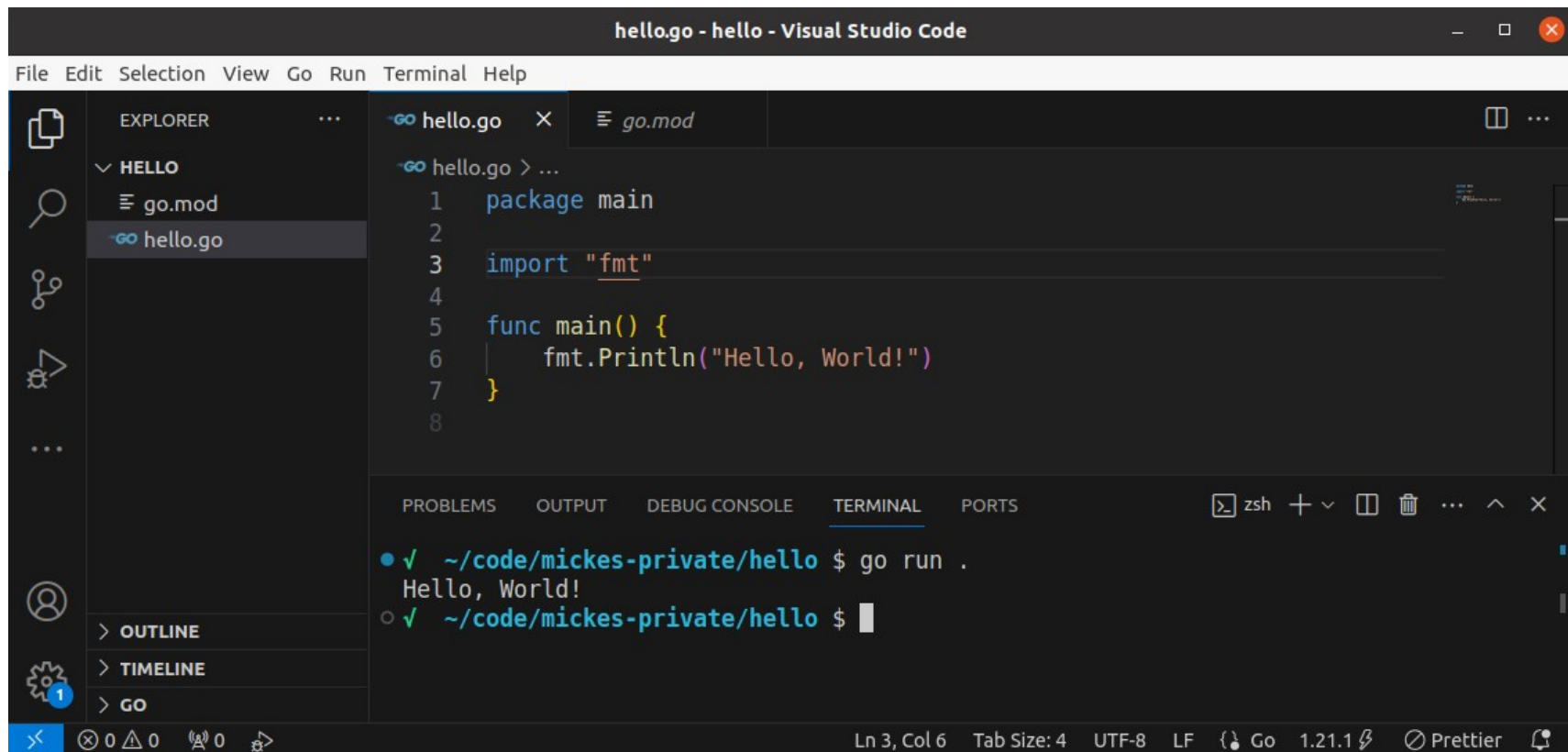
- Create a file "hello.go" in your hello folder and insert the following text

hello.go

```
package main  
import "fmt"  
func main() {  
    fmt.Println("Hello, World!")  
}
```

Hello world

- Run the program in the terminal by:
`$ go run .` (or "go run hello.go" to only include this specific file in the execution)



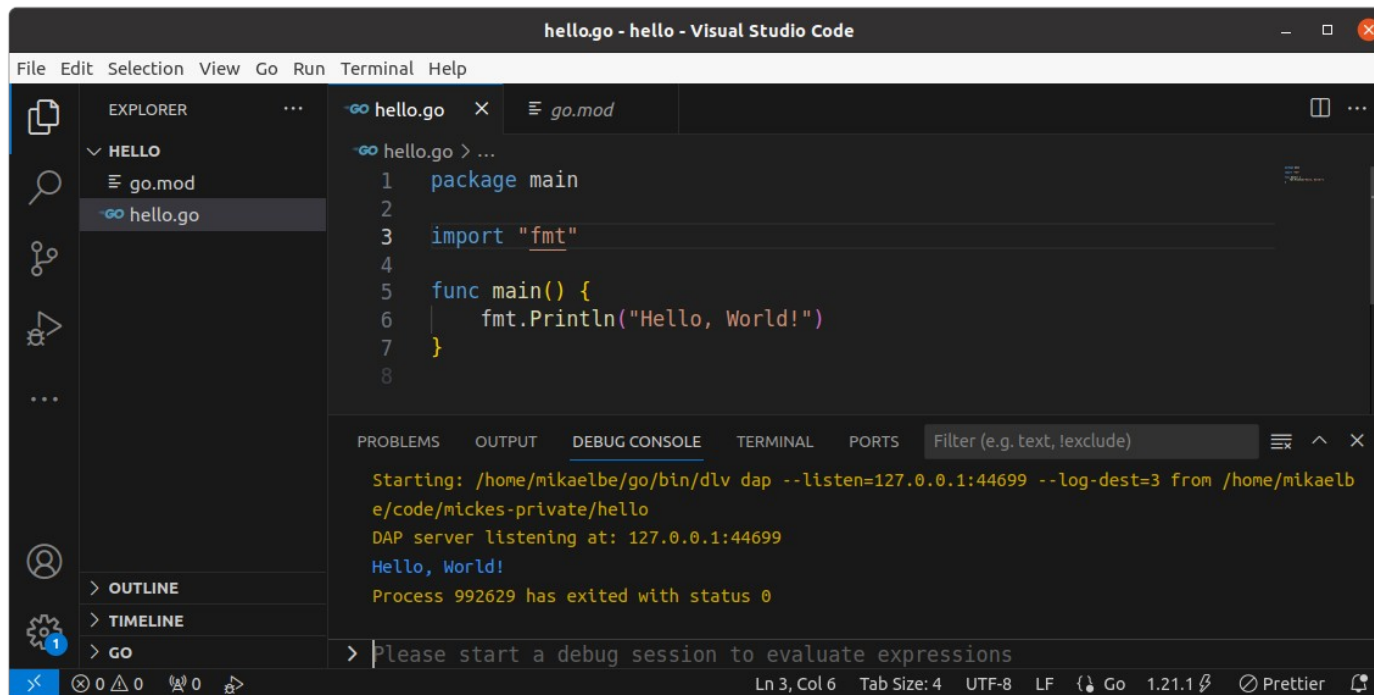
The screenshot shows the Visual Studio Code interface with a Go project. The Explorer sidebar on the left shows a folder named 'HELLO' containing 'go.mod' and 'hello.go'. The main editor displays the 'hello.go' file with the following code:

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     fmt.Println("Hello, World!")
7 }
8
```

Below the editor, the TERMINAL panel is active, showing the command `$ go run .` being executed in a zsh shell. The output is `Hello, World!`. The status bar at the bottom indicates the current position is Line 3, Column 6, with a tab size of 4, UTF-8 encoding, and LF line endings. It also shows the Go version (1.21.1) and the Prettier formatter.

Hello world

- You can also run your program by Ctrl+F5, or by the menu Go → Run Without Debugging. It will then execute the file that is currently in focus. Note, if you run your program this way, it will not send the print to the terminal but to the "DEBUG CONSOLE".



The screenshot shows the Visual Studio Code interface with a Go project named 'hello'. The Explorer sidebar on the left shows the file structure with 'go.mod' and 'hello.go'. The main editor displays the contents of 'hello.go', which is a simple Go program that prints 'Hello, World!'. The interface is running in a debug session, as evidenced by the 'DEBUG CONSOLE' tab being active in the bottom panel. The console output shows the DAP server starting, listening on 127.0.0.1:44699, and then printing 'Hello, World!'. The status bar at the bottom indicates the current file is 'Ln 3, Col 6' and the editor is using 'UTF-8' encoding.

```
hello.go - hello - Visual Studio Code
File Edit Selection View Go Run Terminal Help

EXPLORER
HELLO
  go.mod
  hello.go

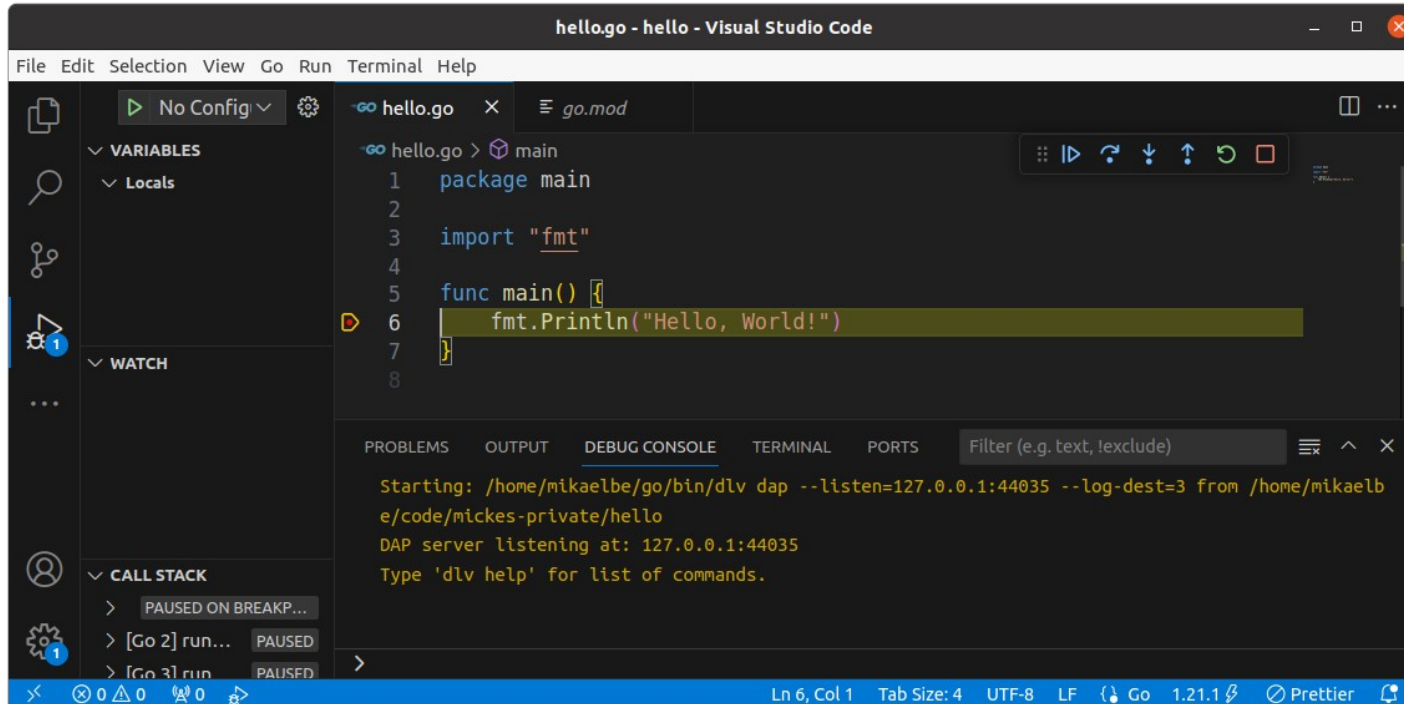
hello.go > ...
1 package main
2
3 import "fmt"
4
5 func main() {
6     fmt.Println("Hello, World!")
7 }
8

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text, texclude)
Starting: /home/mikaelbe/go/bin/dlv dap --listen=127.0.0.1:44699 --log-dest=3 from /home/mikaelbe/code/mickes-private/hello
DAP server listening at: 127.0.0.1:44699
Hello, World!
Process 992629 has exited with status 0
> Please start a debug session to evaluate expressions

Ln 3, Col 6 Tab Size: 4 UTF-8 LF Go 1.21.1 Prettier
```

Debug

- Put a breakpoint by clicking to the left of a code line so a red dot appears. Then press F5, or from the menu choose Go → Start Debugging. The execution will stop at your breakpoint. Note that it is the currently active file that will be executed.



Build



You build your program to an executable from the terminal. Here are some build command examples:

```
// Builds an executable for the OS you are in. It gets the same name as your source file containing main().  
$ go build
```

```
// Define another name of your output executable  
$ go build -o myExecutable
```

```
// Build executable for Windows  
$ GOARCH=amd64 GOOS=windows go build -o ./bin/myProg.exe
```

```
// Build executable for Linux  
$ GOARCH=amd64 GOOS=linux go build -o ./bin/myProg
```

```
// Change path to cmd/Main before building  
$ go -C cmd/Main build
```

```
# Use build flags (Ex of two build flags in a src file: //go:build Mickses || Calles)  
$ Go build -tags Mickses
```

```
# If not using -trimpath the whole absolute path to your files will be visible in logs. This is the path to  
# the files on the computer that you built your executable from. This does not look nice in a prod build.  
$ Go build -trimpath
```

Build



One thing to note is that if you have several source files in the main package, and your main file is using functionality from the other files, you will need to include all of the used files in your build command. But normally you don't specify a single file, like "go build main.go", but instead just do "go build" which will include all files in the current folder. Files that you import from other packages are declared in your main file so those will be included by the compiler automatically.

```
// Build three files  
$ go build main.go helpers.go constants.go
```

```
// Build all files in current folder  
$ go build .
```

```
// Same as above (Note that for the run command you cannot exclude the .)  
$ go build
```

Build



You can overwrite source variable values during the build step. To do this you add the build flag **-ldflags**.

For this to work the variable name must start with a capital letter (public variable).

In the below example we overwrite the "Id" variable, that is located in the example/hello/pkg/helpers package, to 5 and the "Name" variable to "Micke".

```
$ go build -ldflags "-X 'example/hello/pkg/helpers.Id=5' -X 'example/hello/pkg/helpers.Name=Micke'"
```

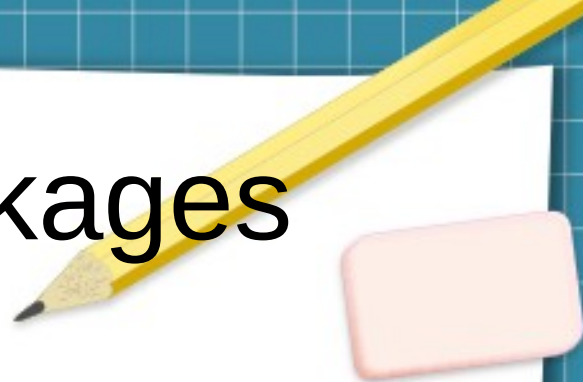
Install



Installera ditt program så du kan köra det direkt från terminalen:

```
$ go install
```

Importing 3rd party packages



Search for packages here: <https://pkg.go.dev/>

Example: <https://pkg.go.dev/rsc.io/quote/v4>

You can see all available functions on its page.

Update your code to use its Go() function.

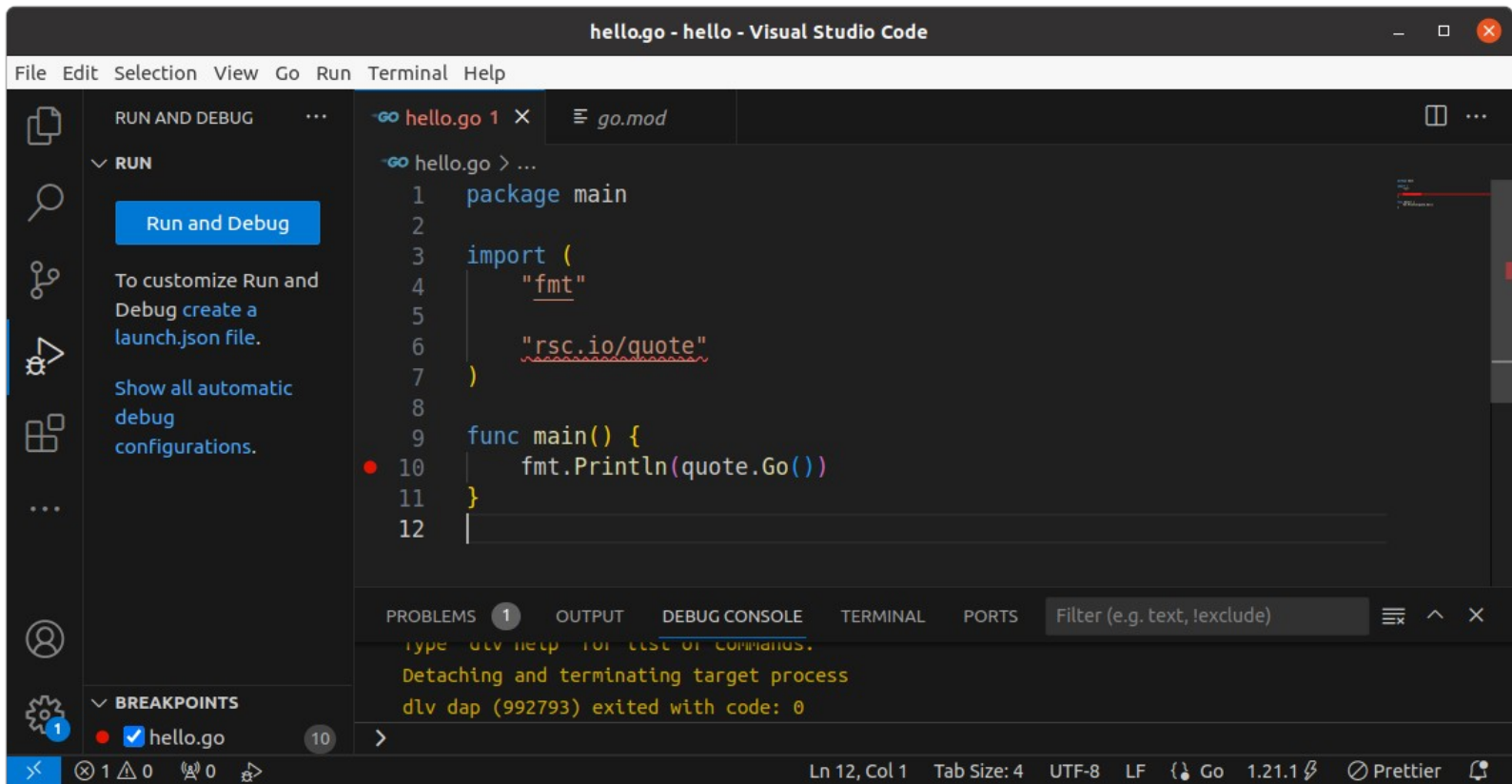
```
package main

import (
    "fmt"
    "rsc.io/quote"
)

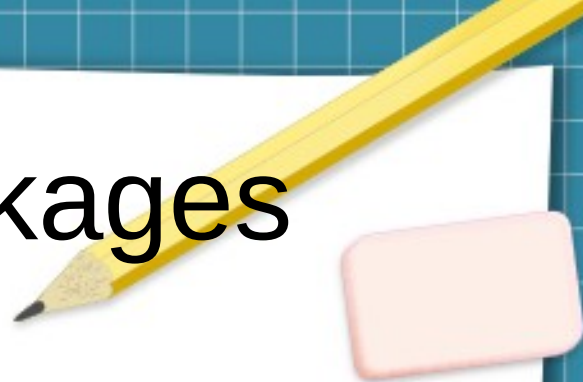
func main() {
    fmt.Println(quote.Go())
}
```

Importing 3rd party packages

In VsCode it would now look like below. As you can see VsCode does not recognize `rsc.io/quote`.



Importing 3rd party packages



To make the wanted packages available in your program you should run the following command in your terminal.

```
$ go mod tidy
```

This will declare your used packages into your **go.mod** file and also create a **go.sum** file, see files on next slide. It will also remove unused packages from both go.mod and go.sum.

After this the functions in your imported packages can be freely used in your code where you imported it.

When you ran go mod tidy, it located and downloaded your wanted rsc.io/quote module, into the module cache. By default, it downloaded the latest version.

It also computed checksums for the downloaded packages and stored those to the go.sum file. This is used during building to verify that the imported packages have not been tampered with since you downloaded them.

Importing 3rd party packages

≡ go.mod

Reset go.mod diagnostics | Run go mod tidy | Create vendor directory

1 module example/hello

2

3 go 1.21.1

4

Check for upgrades | Upgrade transitive dependencies | Upgrade direct dependencies

5 require rsc.io/quote v1.5.2

6

7 require (

8 | golang.org/x/text v0.0.0-20170915032832-14c0d48ead0c // indirect

9 | rsc.io/sampler v1.3.0 // indirect

10)

11

≡ go.sum

1 golang.org/x/text v0.0.0-20170915032832-14c0d48ead0c h1:qg0Y6WgZOaTkIIMiVjBQcw93

2 golang.org/x/text v0.0.0-20170915032832-14c0d48ead0c/go.mod h1:NqM8EU0U14njkJ3fq

3 rsc.io/quote v1.5.2 h1:w5fcysjrx7yqtD/a0+QwRjYZ0KnaM9Uh2b40tElTs3Y=

4 rsc.io/quote v1.5.2/go.mod h1:LzX7hefJvL54yjefDEDHN0NDjII0t9xZLPXsUe+TKr0=

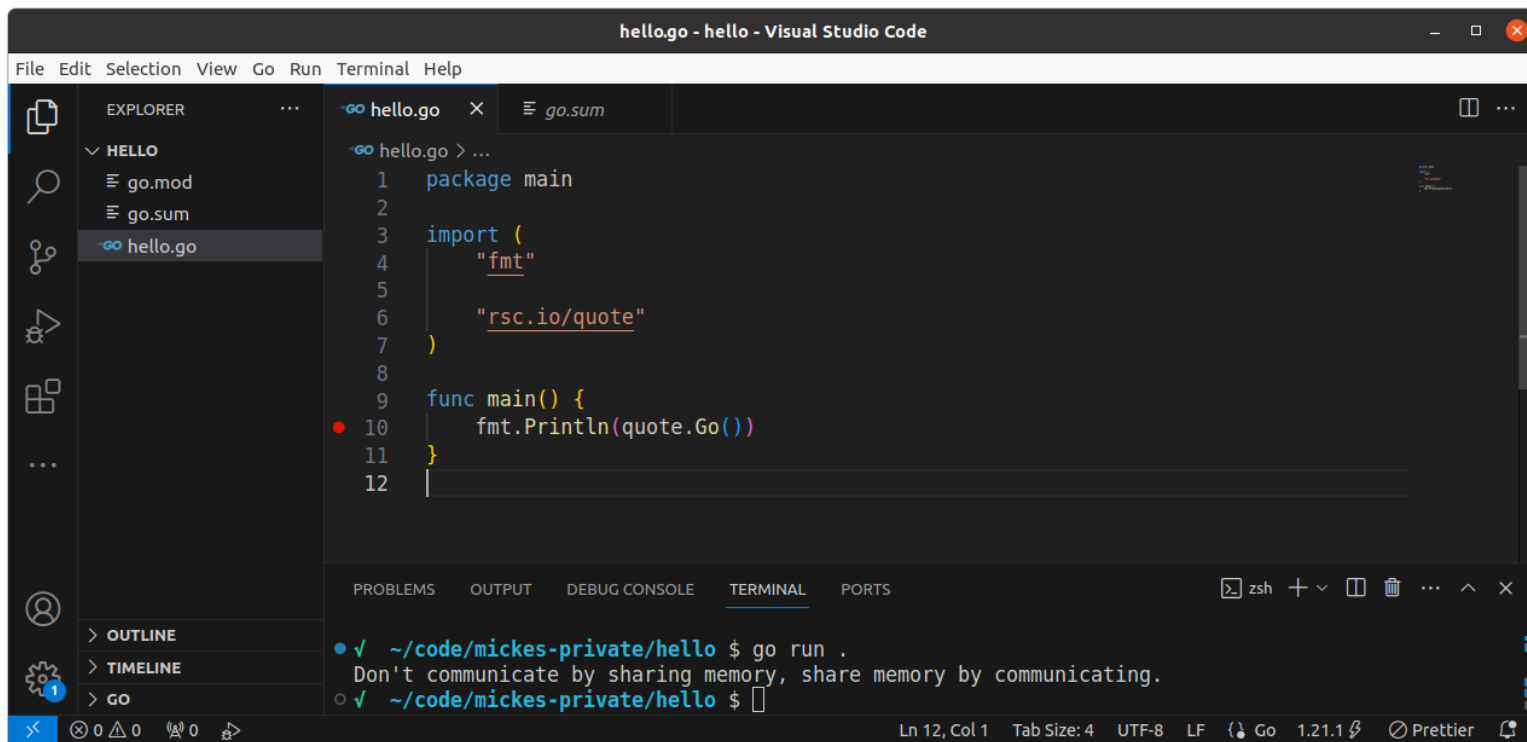
5 rsc.io/sampler v1.3.0 h1:7uVkIFmeBqHfdjD+gZwtXXI+R0DJ2Wc407MPEh/QiW4=

6 rsc.io/sampler v1.3.0/go.mod h1:T1hPZKmBbMNahiBKFy5HrXp6adAjACjK9JXDnKaTXpA=

7

Importing 3rd party packages

Run your new code...



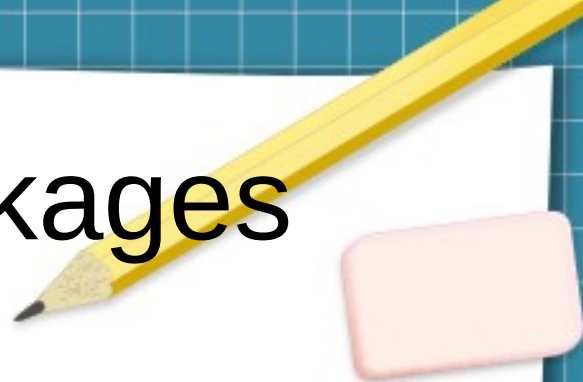
The screenshot shows the Visual Studio Code editor with a Go file named `hello.go`. The code in the editor is as follows:

```
1 package main
2
3 import (
4     "fmt"
5
6     "rsc.io/quote"
7 )
8
9 func main() {
10     fmt.Println(quote.Go())
11 }
12
```

The terminal at the bottom shows the command `go run .` being executed, which results in the output: `Don't communicate by sharing memory, share memory by communicating.`

It just prints: "Don't communicate by sharing memory, share memory by communicating"

Importing 3rd party packages

A yellow pencil is positioned diagonally in the top right corner, pointing towards the bottom left. Below the pencil's tip is a small, rectangular pink eraser.

- One thing that is special with Go is that the compiler does not allow you to import a package that you are not using. If you try to do this the package will be automatically removed when you save your go file.

Subfolders



- To make those demos easier we will now split each demo into subdirectories and call each demo file "main.go". Note that subdirectories shall not contain any space character.

You will then **run** them by the command:

```
go -C <FOLDER> run main.go
```

and **build** with

```
go -C <FOLDER> build main.go
```

Variables



- Go is "statically typed" which means you'll have to declare the types of all variables and the type cannot change. The compiler warns you if you try to assign something of the incorrect type, like an int to a string variable.
- The compiler also warns you of declared but unused variables. You cannot even run a program with a declared unused variable.
- Initialization with ":= " or with "=".
":=" only works within a function scope, outside in global scope you must use "=".

Variables



- `a := "Hello world"`

This lets the Go compiler automatically declare the variable "a" to the correct type and initializes "a" to the string "Hello world" in one line.

- `var a string`

`a = "Hello world"`

Lets you declare the type of "a" and initializes it to the string "Hello world"

- `a, b, c := 1, "Hello world", 3`
- `var a, b, c int = 1, 2, 3`

Data types



- bool
- string
- int, int8, int16, int32, int64
- uint, uint8, uint16, uint32, uint64
- float32, float64
- complex64, complex128
- byte
- rune
- error

Data types



- **Arrays**

Predefined length.

Can contain different values if declared with type "any".

```
var a [3]string = [3]string{"Kalle", "Ada", "Lisa"}
```

```
a = append(a, "Micke") // Does not work for arrays!!!
```

- **Slices**

Dynamic length

Can contain different values if declared with type "any".

```
var a = []int{10, 20, 30, 40}
```

```
a = append(a, 50)
```

Data types



- **Structs**

```
type Employee struct {  
    firstName  string  
    lastName   string  
    age        int  
}
```

```
func (e Employee) printFullName() {  
    fmt.Printf("Employee full name is %s %s", e.firstName, e.lastName)  
}
```

```
func (e *Employee) changeName(name string) {  
    e.firstName = name  
}
```

```
emp1 := Employee{  
    firstName: "Sam",  
    age:      25,  
    lastName: "Anderson",  
}
```

```
emp2 := Employee{"Thomas", "Svensson", 29}
```

```
emp2.printFullName()  
emp2.changeName("Kalle")
```

Data types



- **Maps**

```
make(map[type of key]type of value)
employeeSalary := make(map[string]int)
employeeSalary["Gunnar"] = 12000
```

- **Pointer**

```
a := new(int)
```

```
or
```

```
b := 42
```

```
var a *int = &b
```

```
fmt.Println("The value on the adress a is pointing to is: ", *a)
```

Packages



- Each file of the same package should be located inside a common folder, which preferably has the same name as the package.
- You cannot have two go files of different packages in the same folder.

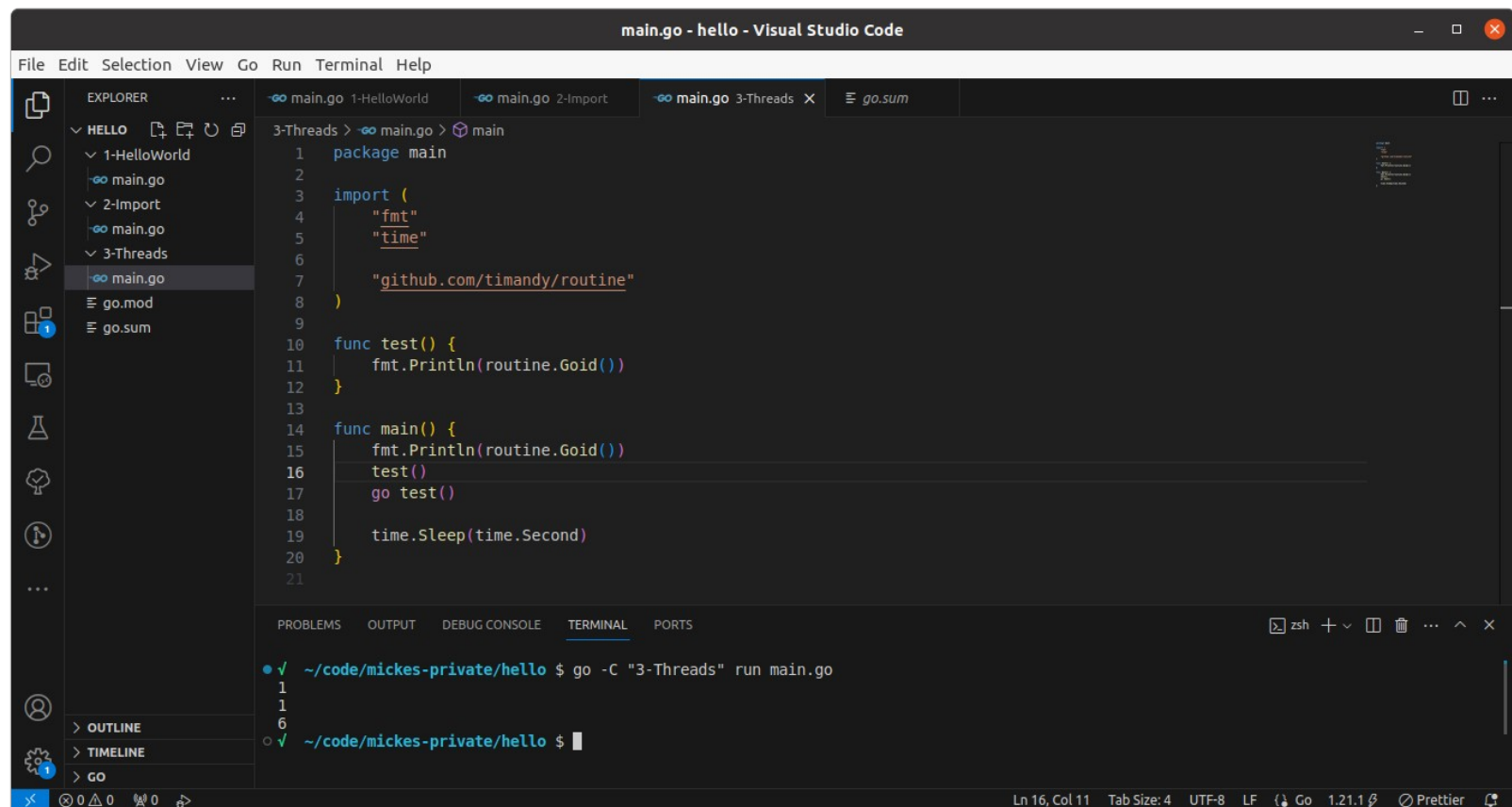
Private/Public



- In Go, the visibility of variables and functions is determined by the capitalization of the first letter of their names.
- This convention applies to both variables and functions.
- Functions and variables, starting with a capital letter, are accessible from other packages (public).
- Functions and variables, starting with a lower case letter, are only accessible from within the same package (private).

Go routines

To run any code in an own thread you just put the word "go" before the code. We call this a goroutine. See example below for calling a function normally and for calling the same function as a goroutine (thread). To demo this we imported the timandy/routine package for printing something similar to a thread id.

A screenshot of the Visual Studio Code editor interface. The Explorer sidebar on the left shows a project named 'HELLO' with three subfolders: '1-HelloWorld', '2-Import', and '3-Threads'. Each folder contains a 'main.go' file. The '3-Threads' folder is selected, and its 'main.go' file is open in the editor. The code in the editor is as follows:

```
1 package main
2
3 import (
4     "fmt"
5     "time"
6
7     "github.com/timandy/routine"
8 )
9
10 func test() {
11     fmt.Println(routine.Goid())
12 }
13
14 func main() {
15     fmt.Println(routine.Goid())
16     test()
17     go test()
18
19     time.Sleep(time.Second)
20 }
21
```

The terminal at the bottom shows the command 'go -C "3-Threads" run main.go' being executed. The output shows three thread IDs: '1', '1', and '6', each on a new line. The status bar at the bottom indicates 'Ln 16, Col 11', 'Tab Size: 4', 'UTF-8', 'LF', 'Go', '1.21.1', and 'Prettier'.

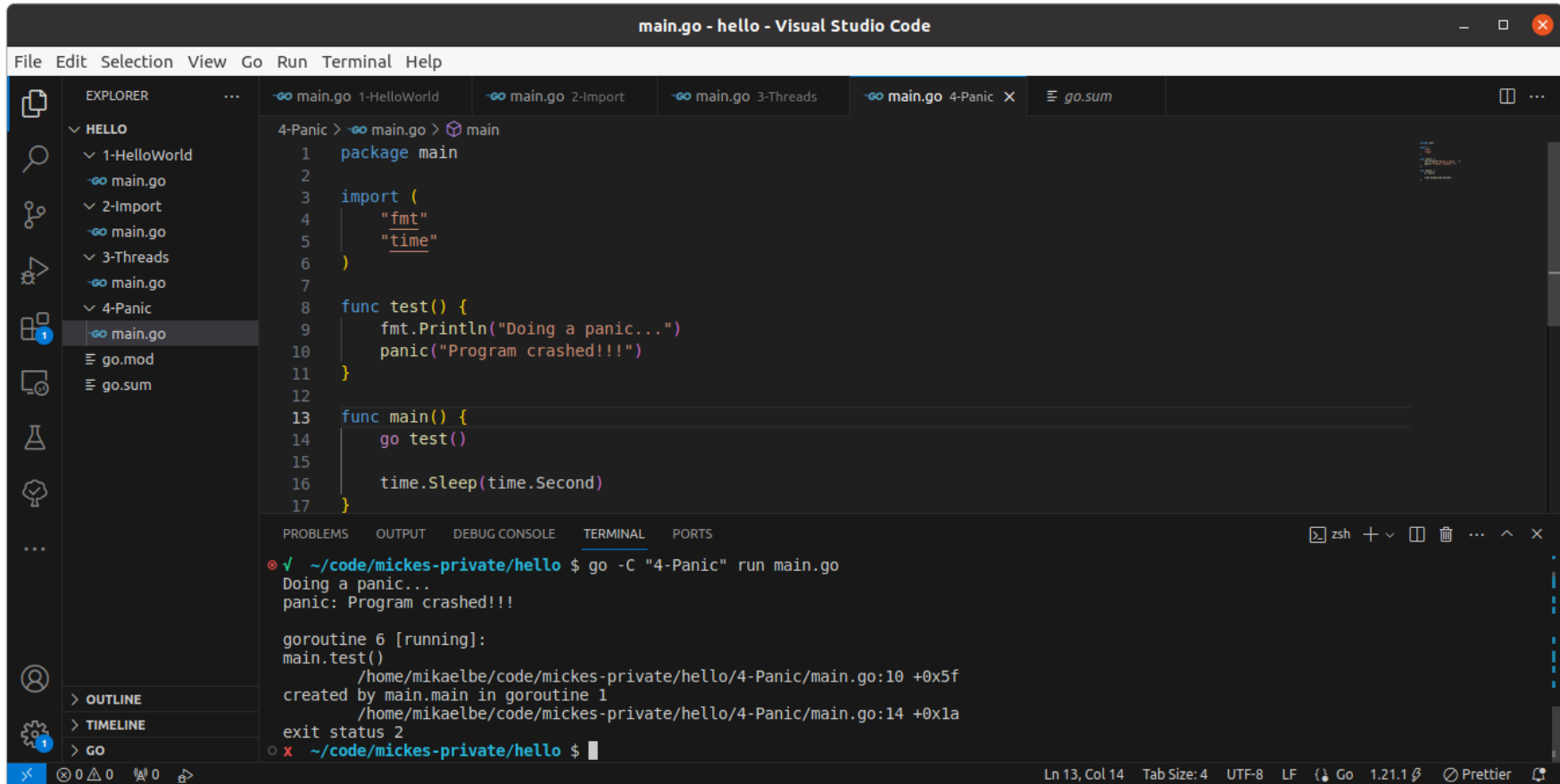
Go routines



- Worth noting from the demo above is that we need to add a sleep to the end of the main function. This is because the main function does not automatically wait for sub goroutines to finish. And if the main function exits, all goroutines will be abruptly terminated directly. So, to ensure the goroutine have time to do its print, we added a sleep.

Panic

- To force the program to crash in Go, we use a built in function called "Panic".



```
main.go - hello - Visual Studio Code
File Edit Selection View Go Run Terminal Help

EXPLORER
HELLO
  1-HelloWorld
  2-Import
  3-Threads
  4-Panic
    main.go
  go.mod
  go.sum

4-Panic > main.go > main
1 package main
2
3 import (
4     "fmt"
5     "time"
6 )
7
8 func test() {
9     fmt.Println("Doing a panic...")
10    panic("Program crashed!!!")
11 }
12
13 func main() {
14     go test()
15
16     time.Sleep(time.Second)
17 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
~/code/mickes-private/hello $ go -C "4-Panic" run main.go
Doing a panic...
panic: Program crashed!!!

goroutine 6 [running]:
main.test()
/home/mikaelbe/code/mickes-private/hello/4-Panic/main.go:10 +0x5f
created by main.main in goroutine 1
/home/mikaelbe/code/mickes-private/hello/4-Panic/main.go:14 +0x1a
exit status 2
~/code/mickes-private/hello $
```

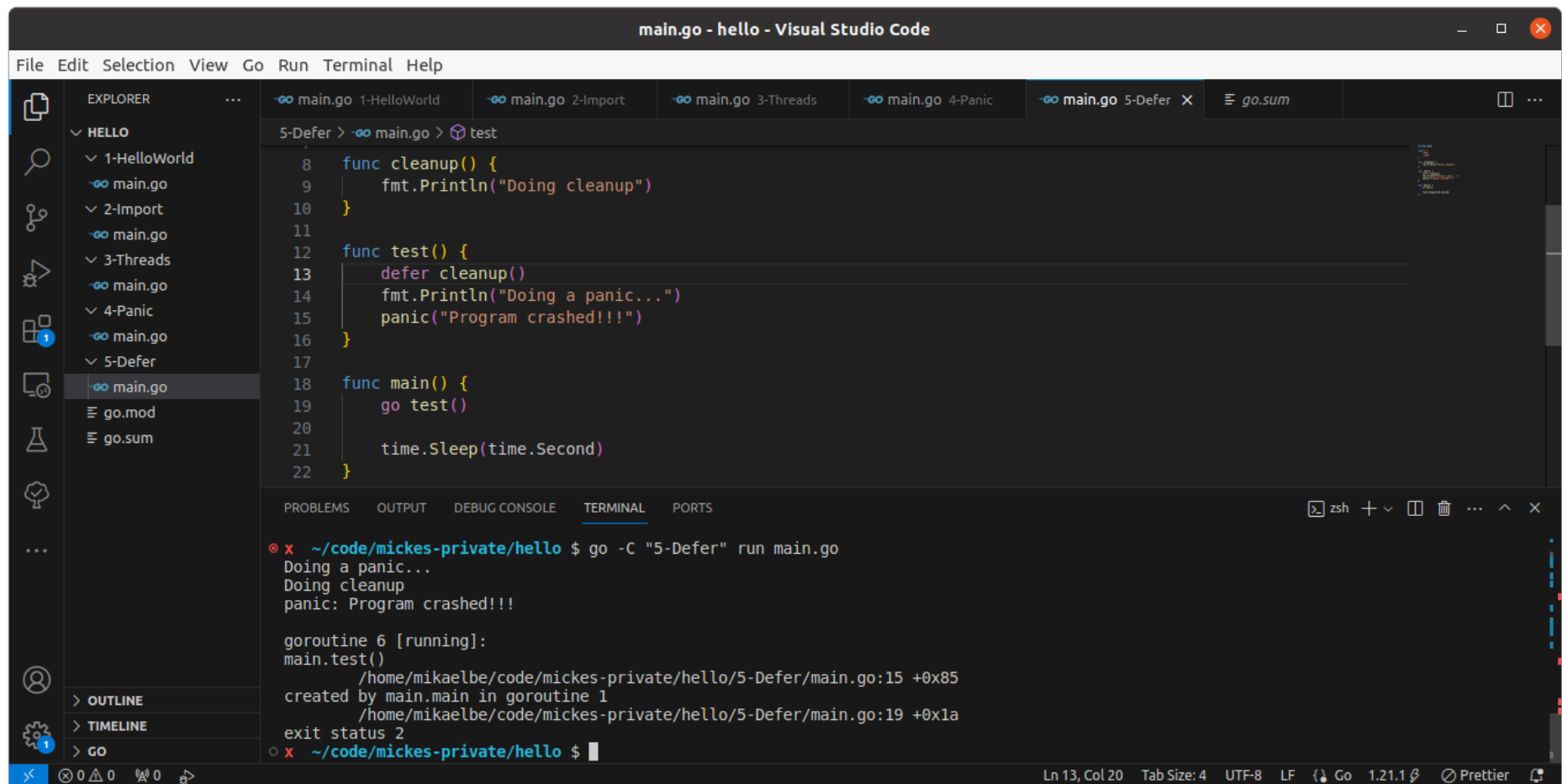
Defer



- Go does not have try/catch.
- But it has "defer"!!
- All defer functions that you declare will be guaranteed to execute at the end of the function it is declared in, even if your function crashes.
- A downside with defer is that only defers occurring in the same goroutine as the crash, will be executed.
- Multiple defer functions can be declared. The last one declared will be executed first, then the others (LIFO).

Defer

- Example of a defer in a goroutine that crashes.



```
main.go - hello - Visual Studio Code
File Edit Selection View Go Run Terminal Help

EXPLORER
HELLO
  1-HelloWorld
  2-Import
  3-Threads
  4-Panic
  5-Defer
  main.go
  go.mod
  go.sum

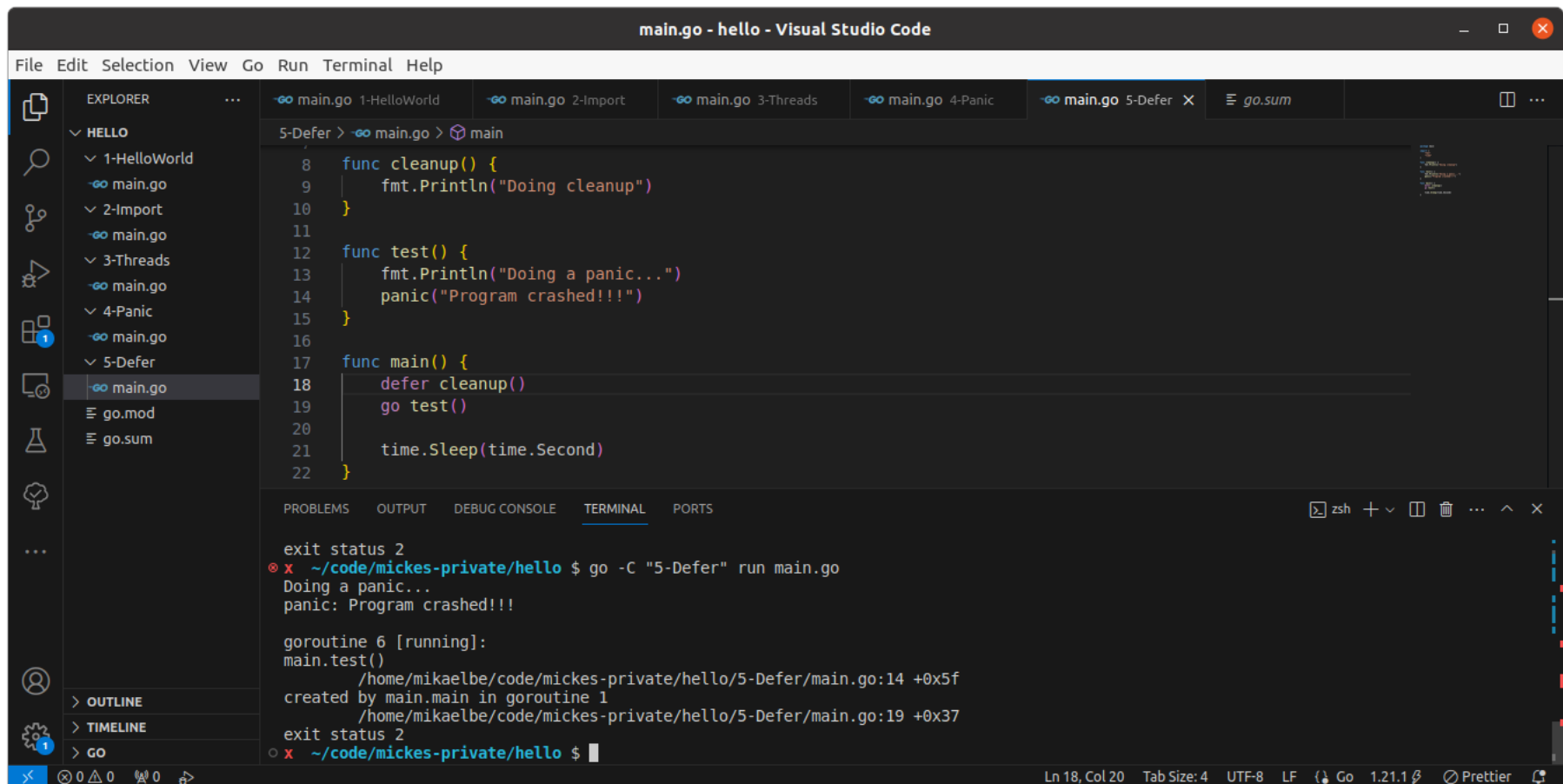
5-Defer > main.go > test
8 func cleanup() {
9     fmt.Println("Doing cleanup")
10 }
11
12 func test() {
13     defer cleanup()
14     fmt.Println("Doing a panic...")
15     panic("Program crashed!!!")
16 }
17
18 func main() {
19     go test()
20
21     time.Sleep(time.Second)
22 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
x ~/code/mickes-private/hello $ go -C "5-Defer" run main.go
Doing a panic...
Doing cleanup
panic: Program crashed!!!

goroutine 6 [running]:
main.test()
/home/mikaelbe/code/mickes-private/hello/5-Defer/main.go:15 +0x85
created by main.main in goroutine 1
/home/mikaelbe/code/mickes-private/hello/5-Defer/main.go:19 +0x1a
exit status 2
x ~/code/mickes-private/hello $
```

Defer

- Example showing that the defer is not executed if a crash happens in another goroutine.



The screenshot shows the Visual Studio Code editor with a Go project named 'main.go - hello'. The Explorer sidebar on the left shows a folder named 'HELLO' containing several files: '1-HelloWorld', '2-Import', '3-Threads', '4-Panic', '5-Defer', 'go.mod', and 'go.sum'. The '5-Defer' file is selected, and its contents are displayed in the editor. The code defines a 'cleanup' function that prints 'Doing cleanup', a 'test' function that prints 'Doing a panic...' and then panics with the message 'Program crashed!!!', and a 'main' function that defers 'cleanup', calls 'test', and sleeps for one second. The TERMINAL panel at the bottom shows the output of running the program: 'exit status 2', 'Doing a panic...', 'panic: Program crashed!!!', and a stack trace for 'goroutine 6 [running]: main.test()'. The stack trace shows the call to 'main.test()' at line 14 and the call to 'main.main()' at line 19. The status bar at the bottom indicates the current position is at line 18, column 20, with a tab size of 4, UTF-8 encoding, and LF line endings.

```
main.go - hello - Visual Studio Code
File Edit Selection View Go Run Terminal Help

EXPLORER
HELLO
  1-HelloWorld
  2-Import
  3-Threads
  4-Panic
  5-Defer
  go.mod
  go.sum

5-Defer > main.go
8 func cleanup() {
9     fmt.Println("Doing cleanup")
10 }
11
12 func test() {
13     fmt.Println("Doing a panic...")
14     panic("Program crashed!!!")
15 }
16
17 func main() {
18     defer cleanup()
19     go test()
20
21     time.Sleep(time.Second)
22 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
exit status 2
x ~/code/mickes-private/hello $ go -C "5-Defer" run main.go
Doing a panic...
panic: Program crashed!!!

goroutine 6 [running]:
main.test()
/home/mikaelbe/code/mickes-private/hello/5-Defer/main.go:14 +0x5f
created by main.main in goroutine 1
/home/mikaelbe/code/mickes-private/hello/5-Defer/main.go:19 +0x37
exit status 2
x ~/code/mickes-private/hello $
```

- Recover from panic

The screenshot shows the Visual Studio Code editor with a Go project named "main.go - hello". The Explorer sidebar on the left shows a file tree with "HELLO" as the root, containing "1-HelloWorld", "2-Import", "3-Threads", "4-Panic", "5-Defer", and "6-Recover". The "6-Recover" folder is selected, showing "main.go".

The main editor displays the code for "6-Recover" in "main.go":

```
8 func catchCrash() {
9     if r := recover(); r != nil {
10         stack := debug.Stack()
11         fmt.Printf("Error: %s\n", r) // Prints the error
12         fmt.Printf("Stacktrace: %s", stack) // Prints the stacktrace
13     } else {
14         fmt.Println("Normal shutdown")
15     }
16 }
17
18 func test() {
19     defer catchCrash()
20     fmt.Println("Doing a panic...")
21     panic("Program crashed!!!")
22 }
23
24 func main() {
25     test()
26     fmt.Println("After crash!!")
27 }
```

The bottom panel shows the TERMINAL output:

```
• ✓ ~/code/mickes-private/hello $ go -C "6-Recover" run main.go
Doing a panic...
Error: Program crashed!!!
Stacktrace:
goroutine 1 [running]:
runtime/debug.Stack()
    /usr/local/go/src/runtime/debug/stack.go:24 +0x5e
main.catchCrash()
    /home/mikaelbe/code/mickes-private/hello/6-Recover/main.go:10 +0x3f
panic({0x483be0?, 0x4b42f8?})
    /usr/local/go/src/runtime/panic.go:914 +0x21f
main.test()
    /home/mikaelbe/code/mickes-private/hello/6-Recover/main.go:21 +0x85
main.main()
    /home/mikaelbe/code/mickes-private/hello/6-Recover/main.go:25 +0x13
After crash!!
• ✓ ~/code/mickes-private/hello $
```

Recover



- If `recover()` is called, the panic is caught and it is not propagated up to `main`.
This means that only the crashing function ends after `defer` and the rest of the program continues.
- If `recover()` is not called, the panic will cause the whole program to exit after the `defer` function is done. The error would then be reported on `stderr`.

Normal error handling



- The Go idiomatic way, to handle errors without exception, is that each function should take care of its own problems and return an error variable along with the normal result.
- When a function returns a value and an error, the caller checks the error to determine if the function completed successfully or if an error occurred. If the error is not nil, it indicates that an error occurred and the returned value should not be used.

Normal error handling



- Example

```
// ReadFile reads a file and returns its content and an error if something goes wrong.
func ReadFile(file string) ([]byte, error) {
    data, err := os.ReadFile(file)
    if err != nil {
        return nil, fmt.Errorf("failed to read file: %w", err)
    }
    return data, nil
}

func main() {
    content, err := ReadFile("example.txt")
    if err != nil {
        fmt.Println("Error:", err)
        return
    }
    fmt.Println("File content:", string(content))
}
```

Channels



- A channel is used to communicate between goroutines and make your code event driven.

```
func main() {  
    // Create a channel to signal completion  
    done := make(chan bool)  
  
    // Start a goroutine that will do some work and then signal completion  
    go func() {  
        fmt.Println("Doing some work...")  
        time.Sleep(2 * time.Second) // Simulate work  
        fmt.Println("Work done!")  
        done <- true // Signal completion  
    }()  
  
    // Wait for the goroutine to signal completion  
    <-done  
    fmt.Println("Main function continues after goroutine completes.")  
}
```

Channels



- `data := <- a` `// read from channel a`
 `a <- data` `// write to channel a`
- Sends and receives to a channel are blocking by default.
- If no receiver → Deadlock → panic.
- You can declare channels to transfer any of the data types, like `bool`, `string`, `int`, `struct`, `map`, `array`, `slice` and even other channels.

Channels



- Wait for several channels

```
func worker(name string, timer chan int) {  
    // Create a new random source with a seed  
    r := rand.New(rand.NewSource(time.Now().UnixNano()))  
    // Generate a random number between 1 and 10  
    rnd := r.Intn(10) + 1  
  
    time.Sleep(time.Second * time.Duration(rnd))  
    timer <- rnd  
}  
  
func main() {  
    a, b, c := make(chan int), make(chan int), make(chan int)  
  
    go worker("Analytics", a)  
    go worker("ACS", b)  
    go worker("IT", c)  
  
    // Wait for every worker to finish  
    worktime1, worktime2, worktime3 := <-a, <-b, <-c  
  
    fmt.Printf("Analytics finished in %ds\n", worktime1)  
    fmt.Printf("ACS finished in %ds\n", worktime2)  
    fmt.Printf("IT finished in %ds\n", worktime3)  
}
```

Select



- An example of a select statement in a goroutine that makes the goroutine close down if it receives either a signal on any channel or a timeout.

```
func main() {  
  
    shutdownChan := make(chan struct{})  
    go func(shutdownChan chan struct{}) {  
        select {  
        case <-shutdownChan:  
            fmt.Println("Shutdown signal received, terminating worker.")  
            // Perform any cleanup or finalization here  
            return  
  
        case <-time.After(10 * time.Second):  
            fmt.Println("Timeout reached, terminating worker.")  
            // Perform any cleanup or finalization here  
            return  
        }  
    }(shutdownChan)  
  
    // Do other stuff here...  
  
    close(shutdownChan) // This will trigger the shutdownChan case in the worker  
}
```

Select



- An example of a work competition.

```
func worker(name string, timer chan int) {
    // Create a new random source with a seed
    r := rand.New(rand.NewSource(time.Now().UnixNano()))
    // Generate a random number between 1 and 10
    rnd := r.Intn(10) + 1

    time.Sleep(time.Second * time.Duration(rnd))
    timer <- rnd
}

func main() {
    a, b, c := make(chan int), make(chan int), make(chan int)

    go worker("Analytics", a)
    go worker("ACS", b)
    go worker("IT", c)

    // Only wait for the first to finish, or timeout
    select {
    case worktime1 := <-a:
        fmt.Printf("Analytics finished in %ds\n", worktime1)
    case worktime2 := <-b:
        fmt.Printf("ACS finished in %ds\n", worktime2)
    case worktime3 := <-c:
        fmt.Printf("IT finished in %ds\n", worktime3)
    case <-time.After(5 * time.Second):
        fmt.Println("Timeout reached, no one finished in time.")
    }
}
```

Waitgroup



- To make the main function wait for sub goroutines it is common to use waitgroups, like below.

```
func worker(name string, wg *sync.WaitGroup) {
    // Create a new random source with a seed
    r := rand.New(rand.NewSource(time.Now().UnixNano()))
    // Generate a random number between 1 and 10
    rnd := r.Intn(10) + 1

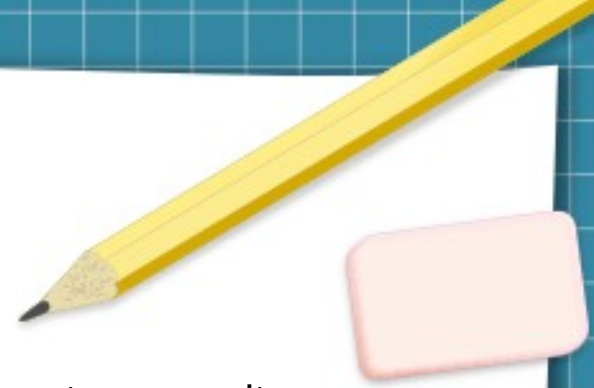
    time.Sleep(time.Second * time.Duration(rnd))

    wg.Done()
}

func main() {
    var wg sync.WaitGroup
    wg.Add(1)
    go worker("Analytics", &wg)
    wg.Add(1)
    go worker("ACS", &wg)
    wg.Add(1)
    go worker("IT", &wg)

    // Wait for every worker to finish
    wg.Wait()
    fmt.Println("All workers have finished")
}
```

Leakage



- A goroutine can not really be leaked in the meaning that it would continue to run on its own when the main function has exited. This is not the case. When the main function exits all goroutines are abruptly terminated automatically, not even their eventual defer functions are executed. They just stop exactly where they are.
- However, if another function than main, starts a goroutine, and that function is terminated, the goroutine will continue to run in the background until all of its code is executed, or the main function ends. This is handled by the go runtime.
- When you hear about goroutine leakage it is probably referring to a goroutine with an endless loop or that listens on a channel that noone will send to. But even those will be terminated when main exits, so they are not leaked outside program scope.

The example below looks like a goroutine leak but everything will be terminated when the main function returns.

```
func main() {  
    go func() {  
        ch := make(chan int)  
        <- ch // Read from channel (no deadlock)  
    }()  
}
```

Context



- If you need to do a graceful shutdown of a goroutine when the main program exits you should preferably use waitgroups in combination with contexts.
- Contexts are often used to signal to all parts in the program that you are about to close down, so each goroutine can close down gracefully.
- When you create a context you always inherit it from a parent context. The top context is the `context.Background()`.
- When you close a context the `Done()` is only called for that context and its sub contexts, not for parent contexts.

Context



```
func worker(ctx context.Context, wg *sync.WaitGroup) {
    defer wg.Done()

    for {
        select {
        case <- ctx.Done():
            fmt.Println("Worker received stop signal, cleaning up!!")
            return
        default:
            fmt.Println("Worker is doing some work...")
            time.Sleep(time.Second * 1)
        }
    }
}

func main() {
    ctx, cancel := context.WithCancel(context.Background())
    var wg sync.WaitGroup

    wg.Add(1)
    go worker(ctx, &wg)

    // Simulate some work on the main thread
    time.Sleep(time.Second * 5)

    cancel() // Signal the goroutine to stop

    wg.Wait()
    fmt.Println("All workers have finished")
}
```

Embed



- Embeds any file into the Go binary at compile time.
- Good for embedding images or text files.
- Good for embedding frontend, like whole react project.
- Access the files from your Go program without needing to read them from the file system at runtime.
- Creates a self-contained application that includes all its necessary resources in one binary.

Embed



Use by adding the following line above a variable:
//go:embed <file or folder>.

(Note the underscore in the import below. This tells the go compiler that embed should be imported even though no exported field from the package is used.)

```
import _ "embed"  
  
//go:embed hello.txt  
var s string
```

The variable can be of type string, []byte, or embed.FS

```
import "embed"  
  
//go:embed image/* template/*  
var content embed.FS
```

Embed



Example of embedding and serving a web page

```
package main

import (
    "embed"
    "io/fs"
    "log"
    "net/http"
)

//go:embed public
var webContent embed.FS
var webContentRoot, _ = fs.Sub(webContent, "public")

func main() {
    mux := http.NewServeMux()
    mux.Handle("/", http.FileServer(http.FS(webContentRoot)))
    err := http.ListenAndServe(":8080", mux)
    if err != nil {
        log.Fatal(err)
    }
}
```

Inheritance

- Composition instead of inheritance
- No classes so inheritance is not really supported.
- But it can be mimiced by **embedded anonymous fields in structs.**



Inheritance



```
type author struct {
    firstName string
    lastName  string
}
func (a author) fullName() string {
    return fmt.Sprintf("%s %s", a.firstName, a.lastName)
}

type blogPost struct {
    title    string
    content  string
    author
}

func (b blogPost) details() {
    fmt.Println("Title: ", b.title)
    fmt.Println("Content: ", b.content)
    fmt.Println("Author: ", b.fullName())
}
```

Polymorphism

- Achieved with the help of interfaces.

```
type Income interface {  
    calculate() int  
    source() string  
}
```

```
type CameraSales struct {  
    cameraType string  
    pricePerUnit int  
    salesAmount int  
}  
  
func (cs CameraSales) calculate() int {  
    return cs.pricePerUnit * cs.salesAmount  
}  
  
func (cb CameraSales) source() string {  
    return cb.cameraType  
}  
  
type AppSales struct {  
    appName string  
    profit int  
}  
  
func (as AppSales) calculate() int {  
    return as.profit  
}  
  
func (as AppSales) source() string {  
    return as.appName  
}
```

Polymorphism



- Achieved with the help of interfaces.

```
func main() {  
    project1 := CameraSales{cameraType: "Wallet", pricePerUnit: 9000, salesAmount: 1000}  
    project2 := AppSales{appName: "DID", profit: 2500000}  
  
    incomes := []Income{project1, project2}  
  
    calculateNetIncome(incomes)  
}  
  
func calculateNetIncome(ic []Income) {  
    var netincome int = 0  
    for _, income := range ic {  
        fmt.Printf("Income From %s = %d\n", income.source(), income.calculate())  
        netincome += income.calculate()  
    }  
    fmt.Printf("Net income of organization = %d\n", netincome)  
}
```

Other



- No **"while"**, only **"for"**.
- **No malloc!!** Memory placement, heap/stack, is decided by the runtime. You should not think about this.
- Json marshal/unmarshal
- Very good info:
<https://golangbot.com/learn-golang-series/>

Thanks...

